

PRODUCT REQUIREMENTS DOCUMENT

CodeSentinel AI

Intelligent Code Review & Security Analysis Platform

Document Version v1.0	Status Draft
Created June 2026	Classification Internal
Product Type AI SaaS Platform	Target Market B2B / B2D

1. Executive Summary

CodeSentinel AI is an AI-powered code review and security analysis platform designed to automatically detect bugs, errors, logic flaws, security vulnerabilities, authentication weaknesses, and privacy risks across an entire codebase. It delivers line-level explanations of every issue found, severity ratings, and actionable remediation guidance — enabling engineering teams to ship more secure, higher-quality software faster.

Problem Statement

Manual code reviews are time-consuming, inconsistent, and often miss subtle security vulnerabilities.

Security audits are expensive and infrequent, leaving codebases exposed between review cycles. Developers lack real-time, context-aware feedback on authentication flaws and data privacy issues. Existing linters cover syntax and style but do not reason about logic errors or security implications.

Solution

An AI system that reads and understands entire codebases — not just individual files — and surfaces bugs, security weaknesses, and privacy risks with precise code locations and fix recommendations.

Supports all major programming languages and integrates into existing developer workflows (CI/CD, IDE, GitHub).

1.1 Key Value Propositions

- **Full-Codebase Analysis:** Scans entire repositories, not just changed files, understanding cross-file dependencies
- **Security-First:** Deep analysis of authentication flows, authorization logic, input validation, and data exposure
- **Privacy Compliance:** Detects GDPR, CCPA, and PII handling violations with specific code references
- **Actionable Output:** Every finding includes severity, affected code location, root cause, and suggested fix
- **Developer-Native:** Integrates with GitHub, GitLab, Bitbucket, VS Code, and CI/CD pipelines

2. Product Vision & Goals

CodeSentinel AI will become the de-facto AI security co-pilot for software engineering teams — a system that every developer trusts to catch what human reviewers miss, before code reaches production.

2.1 Vision Statement

Vision

"To eliminate preventable security breaches and software defects by giving every developer an AI code reviewer that thinks like a security engineer, reasons like a senior architect, and responds in seconds."

2.2 Business Goals

1. Capture 5,000 paying teams within 18 months of launch
2. Achieve <2% false positive rate on critical severity findings
3. Support 15+ programming languages at launch
4. Reduce mean-time-to-detect security vulnerabilities by 80% vs. manual review
5. Become SOC 2 Type II certified within 12 months of launch

2.3 Success Metrics

Metric	Target (M6)	Target (M12)	Target (M18)
Active Teams	500	2,000	5,000
Avg Scan Time (10k LOC)	<30s	<20s	<15s
False Positive Rate	<5%	<3%	<2%
Languages Supported	8	12	15+
NPS Score	40+	50+	60+

3. User Personas

3.1 Primary Personas

Persona 1 — Arjun, Senior Software Engineer (Age 31)

Context: Works at a mid-size SaaS company. Writes 500–800 lines of code per day across Node.js and Python microservices.

Pain: PR reviews take 2–3 days. Security issues found in production cost the team a full sprint to fix.

Needs: Instant feedback in his IDE before pushing code. Clear explanations, not just error codes.

Goal: Ship features fast without introducing vulnerabilities.

Persona 2 — Priya, Engineering Manager (Age 38)

Context: Manages a team of 12 engineers. Responsible for code quality and compliance.

Pain: No visibility into security posture across the codebase. Audits are quarterly and expensive.

Needs: Dashboard showing team-level security trends, risk hotspots, and remediation velocity.

Goal: Reduce security debt and demonstrate compliance to enterprise clients.

Persona 3 — Rahul, DevSecOps Engineer (Age 34)

Context: Owns the CI/CD pipeline and security tooling stack.

Pain: Existing SAST tools have 40%+ false positive rates, causing alert fatigue.

Needs: AI-powered tool with contextual understanding that reduces noise and integrates with GitHub Actions.

Goal: Block vulnerable code from merging without slowing down deployment velocity.

Persona 4 — Sneha, CTO / Technical Co-Founder (Age 42)

Context: Startup with 4 engineers building a fintech product handling sensitive financial and personal data.

Pain: No dedicated security engineer. One breach could kill the company.

Needs: Affordable, automated security review that covers authentication, authorization, and PII handling.

Goal: Build security-by-default culture without hiring a dedicated security team.

4. Core Features & Capabilities

4.1 Full-Codebase Bug Detection

The engine ingests an entire repository and performs multi-pass analysis to identify issues that span files and modules — not just isolated syntax errors.

4.1.1 Bug Categories Detected

- Null pointer dereferences and unhandled undefined/None values
- Off-by-one errors in loops and array indexing
- Race conditions and concurrency bugs (deadlocks, improper mutex usage)
- Memory leaks and resource management failures (unclosed files, DB connections)
- Incorrect exception handling — swallowed errors, overly broad catch blocks
- Type mismatches and implicit type coercions that cause runtime failures
- Infinite loops and unreachable code paths
- Logic errors: incorrect conditional branches, negation bugs, operator precedence issues
- Use of deprecated or removed APIs
- Hardcoded values that should be configurable (URLs, timeouts, limits)

4.1.2 Cross-File Analysis

- Traces function calls and data flow across module boundaries
- Identifies bugs that only manifest when combining logic from multiple files
- Builds a dependency graph of the codebase for holistic understanding

4.2 Security Vulnerability Analysis

The platform performs deep security analysis aligned with OWASP Top 10, CWE/CVE databases, and SANS Top 25. Every finding is mapped to a known vulnerability class.

4.2.1 OWASP Top 10 Coverage

#	OWASP Category	What CodeSentinel Detects
A01	Broken Access Control	Missing authorization checks, IDOR, insecure direct object references, privilege escalation paths
A02	Cryptographic Failures	Weak algorithms (MD5, SHA1), hardcoded keys/secrets, unencrypted sensitive data at rest or in transit
A03	Injection	SQL injection, NoSQL injection, command injection, LDAP injection, template injection
A04	Insecure Design	Missing rate limiting, insecure default configurations, lack of security controls in design patterns

#	OWASP Category	What CodeSentinel Detects
A05	Security Misconfiguration	Debug mode enabled in prod, open S3 buckets, default credentials, verbose error messages
A06	Vulnerable Components	Outdated dependencies with known CVEs, use of deprecated libraries
A07	Authentication Failures	Weak password policies, missing MFA enforcement, insecure session management
A08	Integrity Failures	Insecure deserialization, unsigned code updates, CI/CD pipeline tampering risks
A09	Logging Failures	Missing audit logs for sensitive operations, logging of PII/secrets, insufficient monitoring
A10	SSRF	Unvalidated URLs in server-side requests, missing allowlists for external calls

4.3 Authentication & Authorization Analysis

Authentication and authorization flaws are among the most critical — and most commonly missed — security issues. CodeSentinel performs dedicated deep analysis of auth code.

4.3.1 Authentication Weaknesses Detected

- JWT vulnerabilities: algorithm confusion (none/HS256/RS256 swap), missing expiry validation, weak secret keys
- Session fixation and session hijacking risks
- Insecure password hashing: MD5, SHA1, unsalted hashes, missing bcrypt/argon2
- Missing account lockout after failed login attempts (brute force risk)
- Hardcoded credentials and API keys in source code or config files
- OAuth misconfigurations: open redirects, state parameter not validated, token leakage
- Missing HTTPS enforcement for authentication endpoints
- Remember-me tokens stored insecurely or with infinite expiry

4.3.2 Authorization Weaknesses Detected

- Missing authorization checks on API endpoints (unauthenticated access to protected resources)
- Horizontal privilege escalation: User A can access User B's data
- Vertical privilege escalation: Regular user can perform admin actions
- Insecure Direct Object References (IDOR): predictable IDs without access checks
- Role-based access control (RBAC) logic errors
- Missing ownership validation before update/delete operations

4.4 Privacy & Data Protection Analysis

4.4.1 PII Detection & Handling

- Identifies code paths where PII (names, emails, phone numbers, financial data) is handled
- Detects PII being logged, exposed in API responses, or stored unencrypted
- Flags missing data masking/redaction in logs and error messages
- Identifies missing consent mechanisms for data collection

4.4.2 Compliance Analysis

- GDPR: Right to erasure implementation, consent logging, data minimization violations
- CCPA: Sale of personal data without opt-out, missing privacy notices in code
- HIPAA: PHI exposure in logs, missing encryption for health data at rest
- PCI-DSS: Card data in logs, missing tokenization, insufficient access controls

5. Findings & Severity System

Every issue found by CodeSentinel is assigned a severity level, category, CWE mapping, affected code location, root cause explanation, and remediation guidance.

5.1 Severity Classification

Severity	Definition & Response SLA
CRITICAL	Immediate threat to confidentiality, integrity, or availability. Exploitable with low effort. Examples: SQL injection, hardcoded passwords, unauthenticated admin endpoints. Block merge. Fix before deployment.
HIGH	Significant security risk or data loss potential. Examples: broken access control, weak JWT implementation, unencrypted PII. Fix within current sprint.
MEDIUM	Moderate risk, exploitable under specific conditions. Examples: missing rate limiting, weak session config, verbose error messages. Fix within 2 sprints.
LOW	Best practice violations, minor risks. Examples: missing security headers, logging non-sensitive fields, unused dependencies. Fix within current quarter.
INFO	Informational findings: code quality improvements, refactoring suggestions, performance notes. Address as time permits.

5.2 Finding Report Structure

Each finding generated by CodeSentinel contains the following structured fields:

Sample Finding: SQL Injection Vulnerability
ID: CS-2024-0041
Severity: CRITICAL
Category: Injection (OWASP A03)
CWE: CWE-89 — Improper Neutralization of Special Elements used in SQL Command
File: src/api/users/controller.js | Line: 47
Function: getUserById()

Description: User-supplied input `req.params.id` is concatenated directly into a SQL query string without parameterization or sanitization. An attacker can manipulate this parameter to extract, modify, or delete any data in the database.

Vulnerable Code: `const query = `SELECT * FROM users WHERE id = ${req.params.id}`;`

Recommended Fix: Use parameterized queries or a prepared statement:
`const query = 'SELECT * FROM users WHERE id = ?';`
`db.execute(query, [req.params.id]);`

References: CWE-89, OWASP Testing Guide v4.2 — OTG-INPVAL-005

6. Technical Architecture

6.1 System Components

Ingestion Layer

Git repository cloning and file tree indexing
Language detection and tokenization pipeline
Dependency manifest parsing (package.json, requirements.txt, pom.xml, go.mod)
Chunking strategy: semantic chunking by function/class boundaries, not arbitrary token windows
Incremental scan support: only re-analyze changed files + their dependents

Analysis Engine

Multi-agent LLM architecture: specialist agents for bugs, security, privacy, and auth
AST (Abstract Syntax Tree) analysis for structural understanding of code
Static data flow analysis: tracks tainted inputs from source to sink
Cross-file context window: maintains dependency graph for multi-file reasoning
RAG (Retrieval-Augmented Generation): retrieves relevant CVEs, CWEs, and code patterns
Confidence scoring: each finding has a confidence % based on evidence strength

Output & Integration Layer

Structured findings in JSON, SARIF 2.1 format (industry standard for security tools)
GitHub / GitLab / Bitbucket PR comment injection — findings posted as inline review comments
VS Code extension: real-time feedback as code is written
CI/CD webhook: fail builds on CRITICAL findings, warn on HIGH
Dashboard API: aggregated metrics for team and org-level views
Slack / Teams notifications for new critical findings

6.2 AI Model Strategy

- **Primary Analysis Model:** Fine-tuned code LLM (based on CodeLlama or DeepSeek Coder) trained on vulnerability datasets
- **Security Specialist:** Separate model trained on CVE/CWE databases and security audit reports
- **Context Management:** Hierarchical summarization for large codebases exceeding context window limits
- **False Positive Reduction:** Ensemble approach: multiple passes with different prompting strategies, findings require agreement

- **Self-Hosted Option:** Enterprise tier: run the model on customer's own infrastructure for air-gapped environments

6.3 Language Support

Launch (v1.0)	Phase 2 (v1.5)	Phase 3 (v2.0)	Future
• JavaScript / TypeScript • Node.js • Python • Java / Kotlin	• Go • Ruby on Rails • PHP • Rust	• C / C++ • Swift / Objective-C • Kotlin Multiplatform • Scala	• Solidity (Web3) • R • COBOL • Terraform / IaC

7. User Experience & Interface

7.1 Core User Flows

7.1.1 First-Time Onboarding

1. User signs up with GitHub/GitLab OAuth
2. Selects repositories to connect (org-level or individual repos)
3. CodeSentinel runs initial full-codebase scan — shows progress indicator with file count and estimated completion
4. Dashboard populates with findings organized by severity, category, and file
5. Email summary sent with top 5 critical findings and quick-fix links

7.1.2 PR Review Flow

6. Developer opens a pull request on GitHub
7. CodeSentinel GitHub Action triggers automatically
8. Scan runs on diff + affected dependency files
9. Inline comments posted on PR with finding details and suggested fix
10. PR Check status set: Pass (no new Critical/High), Warning (new Medium), Fail (new Critical/High)
11. Developer addresses findings, pushes new commit, scan re-runs

7.2 Dashboard Screens

7.2.1 Overview Dashboard

- Security Score (0–100): composite score across all findings, trended over time
- Findings heatmap: files ranked by density of critical issues
- Severity breakdown donut chart
- Recent activity feed: new findings, resolved issues, scan history
- Top 5 most-vulnerable files

7.2.2 Findings Explorer

- Filterable table: by severity, category, file, language, date, assignee
- Full finding detail panel: description, code snippet, fix suggestion, CVE links
- Assign findings to team members
- Mark as: Open, In Progress, Resolved, Won't Fix (with justification required for Won't Fix)
- Bulk actions: export to CSV/JIRA, bulk-assign, bulk-resolve

7.2.3 Repository Health View

- Per-repository security score and trend line

- Coverage: % of codebase analyzed
- Language breakdown and LOC count
- Dependency vulnerability summary (from manifest analysis)
- Last scan date and scan history

8. Integrations

Integration	Type	Capabilities
GitHub	SCM + CI/CD	PR comments, status checks, GitHub Actions, OAuth login, repo access
GitLab	SCM + CI/CD	MR comments, pipeline integration, GitLab CI, OAuth, self-hosted support
Bitbucket	SCM	PR comments, pipeline triggers, OAuth login
VS Code	IDE Extension	Real-time inline warnings, quick-fix suggestions, finding panel
JetBrains IDEs	IDE Plugin	IntelliJ, WebStorm, PyCharm — inline annotations and sidebar
Jira	Issue Tracker	Auto-create tickets for Critical/High findings with full context
Slack / Teams	Notifications	Instant alerts for new critical findings, daily digest, scan complete
Webhook API	Custom	POST findings to any endpoint; integrate with custom dashboards
SARIF Export	Standard Format	Import findings into any SARIF-compatible security toolchain

9. Pricing & Packaging

Developer (Free)	Team (\$29/month)	Business (\$99/month)	Enterprise (Custom)						
1 repo Up to 5 repos Unlimited repos Unlimited repos	1 user Up to 10 users Up to 50 users Unlimited users	100 scans/month 500 scans/month Unlimited scans Unlimited scans	Bug detection only Full security scan Full security scan Full security scan	— PR integration PR + CI/CD integration PR + CI/CD integration	— — Jira/Slack integration All integrations	— — — Self-hosted model option	— — — SSO + SAML	— — — SOC 2 report, custom SLA	Community support Email support Priority support Dedicated CSM

10. Development Roadmap

Phase 1 — Foundation (Months 1–3)

Core analysis engine: JavaScript, TypeScript, Python support
Bug detection and OWASP Top 10 security scanning
GitHub integration (PR comments + GitHub Actions)
Web dashboard: findings explorer + overview
Authentication and team management
Basic API for external integrations
Deliverable: Private Beta with 50 pilot teams

Phase 2 — Expansion (Months 4–6)

Add Java, Go, Ruby language support
Privacy analysis module: PII detection, GDPR/CCPA checks
Authentication & authorization deep analysis
VS Code extension launch
GitLab + Bitbucket integrations
Jira and Slack integrations
Pricing tiers launched: Developer, Team, Business
Deliverable: Public Launch

Phase 3 — Scale & Intelligence (Months 7–12)

C/C++, PHP, Swift language support
JetBrains IDE plugin
Advanced ML: custom model fine-tuning on customer codebases to reduce false positives
Team analytics: trend tracking, remediation velocity, security score over time
Dependency vulnerability scanning (SCA)
SOC 2 Type II audit initiated
Enterprise tier: SSO, self-hosted model option
Deliverable: 2,000 paying teams

Phase 4 — Enterprise & Platform (Months 13–18)

Remaining languages: Rust, Kotlin, Scala, Solidity
AI auto-fix: generate code patches, not just suggestions
Policy engine: custom rules and organization-specific security policies
Compliance reports: auto-generated GDPR, PCI-DSS, SOC 2 evidence packages
SOC 2 Type II certification complete
API marketplace: third-party plugins and integrations

Deliverable: 5,000 paying teams, Enterprise certification

11. Security, Privacy & Compliance

11.1 Platform Security (How We Protect Customer Code)

- All code transmitted over TLS 1.3; at-rest encryption with AES-256
- Code is never stored permanently: repos are scanned in ephemeral containers and wiped after analysis
- No training on customer code without explicit opt-in
- SOC 2 Type II certification roadmap (Month 12)
- Penetration testing by third party every 6 months
- Bug bounty program at launch

11.2 Access Controls

- Multi-factor authentication (MFA) enforced for all accounts
- Role-based access control: Admin, Security Lead, Developer, Read-Only
- Audit logs: every user action logged with timestamp, user ID, and IP
- SSO / SAML 2.0 support for Enterprise tier

11.3 Data Residency

- US region (default): AWS us-east-1
- EU region: AWS eu-central-1 (GDPR compliance for EU customers)
- Customer-selectable data residency in Enterprise tier
- Self-hosted deployment option for air-gapped Enterprise environments

12. Non-Functional Requirements

12.1 Performance

- **Scan Speed Target:** 10,000 LOC in under 30 seconds (Phase 1), under 15 seconds (Phase 3)
- **Large Codebase:** 1M+ LOC repositories supported via incremental scanning and parallel processing
- **Concurrent Scans:** Support 1,000 simultaneous scan jobs at launch; scale to 10,000 by Month 12
- **API Response Time:** p95 < 200ms for dashboard API calls; p99 < 500ms

12.2 Reliability & Availability

- **Uptime SLA:** 99.9% for Team/Business; 99.95% for Enterprise
- **Scan Failure Rate:** < 0.1% of scans fail due to platform error
- **Recovery Time Objective (RTO):** < 1 hour for major incidents
- **Recovery Point Objective (RPO):** < 15 minutes (findings data)

12.3 Scalability

- **Horizontal Scaling:** Stateless scan workers auto-scaled via Kubernetes based on queue depth
- **Database:** PostgreSQL with read replicas; findings data partitioned by org ID
- **Queue:** Redis/SQS-based job queue with dead-letter handling and retry logic
- **CDN:** Static dashboard assets served via CloudFront with global edge caching

12.4 Accuracy

- **False Positive Rate Target:** < 5% at launch, < 2% by Month 12
- **False Negative Rate Target:** < 10% for OWASP Top 10 vulnerabilities
- **Confidence Scoring:** All findings include AI confidence score (0–100%)
- **Human Review Flag:** Findings below 70% confidence are flagged for developer verification

13. Risks & Mitigations

Risk	Severity	Mitigation
High false positive rate causes developer distrust	HIGH	Ensemble model approach, confidence scoring, continuous feedback loop from user accept/reject actions
Customer code security breach on our platform	CRITICAL	Ephemeral scan containers, zero persistent code storage, SOC 2 audit, pen testing, encryption at rest and in transit
LLM hallucination causing incorrect findings	HIGH	AST-grounded analysis, deterministic checks for known patterns, confidence thresholds, user feedback mechanism
Competitor with similar feature set (GitHub Copilot, Snyk)	MEDIUM	Differentiate on full-codebase reasoning vs. file-level scanning, auth/privacy specialization, UX quality
Regulatory changes in AI usage of code	MEDIUM	Strict no-training-on-customer-code policy, data residency options, contractual protections
Scan performance degradation at scale	MEDIUM	Load testing at 10x expected volume before launch, auto-scaling architecture, queue-based job processing
Enterprise sales cycle too long to hit revenue targets	MEDIUM	Strong PLG (product-led growth) with free tier, Team tier as self-serve entry point, enterprise for upsell only

14. Open Questions & Decisions Required

12. Build vs. Buy for base LLM: Fine-tune open-source (CodeLlama) vs. license a foundation model vs. use OpenAI/Anthropic API?
13. Monorepo vs. microservices for backend: affects team structure and deployment complexity
14. Scan isolation approach: Docker containers vs. microVMs (Firecracker) vs. WASM sandboxes?
15. How to handle monorepos with 10M+ LOC: full scan prohibitively expensive; smart subsetting required
16. Feedback loop design: how do we learn from developer accept/reject actions without compromising privacy?
17. Go-to-market: developer-led (free tier, PLG) vs. top-down enterprise sales first?
18. On-premise vs. cloud-only: when to invest in self-hosted deployment option?